

Business Intelligence & Agentic AI

Übungsblatt zu Week 11 - 14 Aufgaben für BAIs

Schwerpunkt: BI-Architektur, Agenten, RAG, Governance, Evaluation und einfache Python-Prototypen

Ausgangslage und Arbeitsauftrag

Dieses Übungsblatt vertieft die Inhalte der Week-11-Slides zu Business Intelligence und Agentic AI. Im Zentrum steht die Frage, wie sich Business Intelligence von der reinen Darstellung von Kennzahlen zu einem aktiven, nachvollziehbaren Entscheidungsprozess entwickelt. Die Aufgaben verlangen deshalb nicht nur Begriffswissen, sondern die Anwendung auf konkrete Unternehmenssituationen: Daten müssen eingeordnet, KPIs fachlich definiert, Agenten-Workflows entworfen, Risiken reflektiert und einfache Prototypen in Python umgesetzt werden.

Bearbeiten Sie die Aufgaben so, dass Ihre Lösungen auch für eine fachliche Führungskraft nachvollziehbar wären. Begründen Sie Annahmen, nennen Sie Unsicherheiten und unterscheiden Sie klar zwischen Daten, Interpretation, Empfehlung und Entscheidung. Bei den Python-Aufgaben genügt bewusst einfacher Code ohne externe LLM-API; wichtig ist, dass Planung, Tool-Nutzung, Kontext, Bewertung und menschliche Kontrolle im Kleinen sichtbar werden.

Die Punktzahlen sind als Lernhilfe gedacht. So kann man sich selber überprüfen. ;-)

Viel Spass beim Knobeln...!!!

Übersicht

Nr.	Thema	Leistung	Punkte
1	Vom Dashboard zur Entscheidung	Konzept und Empfehlung	6
2	5-Layer-BI-Architektur	Architekturtransfer	7
3	Chatbot, Copilot oder Agent	Vergleich und Use Cases	6
4	Agentischer BI-Workflow	Ursachenanalyse	8
5	Python: einfacher Agent-Loop	Coding	9
6	Tool-Design fuer BI-Agenten	Schnittstellen und Sicherheit	7
7	RAG versus Agentic RAG	Analyse und Anwendung	7
8	Python: KPI-Anomalie-Agent	Coding	10
9	Semantic Layer und KPI-Governance	KPI-Semantik	7
10	Human-in-the-Loop und Verantwortung	Policy-Matrix	7
11	Single-Agent oder Multi-Agent	Architekturentscheid	7
12	Python: Mini-Retrieval-Agent	Coding	10
13	Evaluation, Monitoring und stille Fehler	Qualitaetssicherung	7
14	Python: Task-Agent Terminplanung	Coding	12

Aufgabe 1: Vom Dashboard zur Entscheidung

Umfang: 6 Punkte

Abgabe: ca. 0.5 bis 1 Seite mit Problem, Daten, KPIs, Analyse, Empfehlung und Verantwortlichkeit

Wählen Sie ein konkretes Unternehmensproblem, bei dem ein klassisches Dashboard allein nicht mehr genügt. Geeignete Beispiele sind ein Umsatzrückgang in einer Region, eine sinkende Conversion Rate im Webshop, steigende Supportkosten, Abwanderung von Kunden oder eine unerwartete Verschlechterung der Marge.

Beschreiben Sie zuerst die Ausgangslage so präzise, dass klar wird, wer entscheiden muss und warum die Entscheidung zeitkritisch oder wirtschaftlich relevant ist. Führen Sie anschliessend aus, welche Datenquellen benötigt werden, welche KPIs berechnet werden sollten und welche Auffälligkeiten ein BI-AI-System automatisch erkennen müsste. Erweitern Sie die Analyse um mögliche Ursachen, eine einfache Prognoseidee und eine konkrete Handlungsempfehlung.

Schliessen Sie mit einer kurzen Reflexion ab: Welche Entscheidung kann das System vorbereiten, und welche Entscheidung muss weiterhin beim Menschen bleiben?

Aufgabe 2: Die 5-Layer-BI-Architektur anwenden

Umfang: 7 Punkte

Abgabe: Tabelle mit fünf Layern plus Begründungstext

Entwerfen Sie für ein selbst gewähltes Unternehmen eine moderne BI-Architektur nach dem 5-Layer-Prinzip. Sie können zum Beispiel einen Onlinehändler, eine Bank, ein Spital, eine Hochschule, einen Industriebetrieb oder eine öffentliche Verwaltung wählen.

Ordnen Sie jedem Layer mindestens drei konkrete Elemente zu: Datenquellen, Data Platform, Semantic Layer, AI Analytics Layer und Decision Layer. Verwenden Sie keine abstrakten Begriffe allein, sondern benennen Sie konkrete Daten, Systeme, Kennzahlen, Modelle oder Entscheidungsartefakte. Beispiel: Statt nur 'CRM' zu schreiben, erklären Sie, welche CRM-Daten für die Analyse relevant sind.

Beschreiben Sie danach, an welcher Stelle ein Agent sinnvoll eingesetzt wird. Begründen Sie ebenso, wo ein Agent nicht oder nur eingeschränkt eingesetzt werden sollte, etwa wegen Datenqualität, fehlender Semantik, Berechtigungen oder hoher Entscheidungsrisiken.

Aufgabe 3: Chatbot, Copilot oder Agent?

Umfang: 6 Punkte

Abgabe: Vergleichstabelle plus drei kurze Use-Case-Beschreibungen

Vergleichen Sie Chatbot, Copilot und Agent im Kontext von Business Intelligence. Arbeiten Sie heraus, wie sich die drei Formen hinsichtlich Autonomie, Tool-Nutzung, Memory, Risiko, Nachvollziehbarkeit und BI-Nutzen unterscheiden.

Formulieren Sie für jede Form einen passenden BI-Use-Case. Der Chatbot kann beispielsweise einfache Fragen zu einem Report beantworten, der Copilot kann bei der Interpretation eines Dashboards helfen, und ein Agent kann eine mehrstufige Analyse mit Tool-Aufrufen und Ergebnisprüfung durchführen.

Achten Sie darauf, nicht nur Funktionen aufzuzählen. Erklären Sie jeweils, welche Rolle der Mensch einnimmt, welche Aufgaben das System selbstständig übernimmt und welche Grenzen im praktischen Einsatz entstehen.

Aufgabe 4: Agentischer BI-Workflow: Ursachenanalyse

Umfang: 8 Punkte

Abgabe: Ablaufdiagramm oder nummerierte Prozessbeschreibung mit Management-Notiz

Die Conversion Rate eines Onlineshops sinkt in Woche 18 deutlich. Entwickeln Sie einen agentischen Workflow, der diese Anomalie untersucht und eine Management-Notiz vorbereitet.

Der Workflow soll mindestens sechs Schritte enthalten. Berücksichtigen Sie Kanal, Gerätetyp, Produktgruppe, Kampagnen, Ladezeiten und Support-Tickets. Beschreiben Sie für jeden Schritt, welche Information gesucht wird, welches Tool oder welche Datenquelle verwendet wird und wie das Zwischenergebnis bewertet wird.

Am Ende soll der Agent nicht einfach eine Zahl ausgeben. Er soll eine plausible Ursache priorisieren, alternative Erklärungen nennen, Unsicherheiten offenlegen und eine konkrete nächste Massnahme vorschlagen. Formulieren Sie zum Schluss ein Beispiel für die Management-Notiz in drei bis fünf Sätzen.

Aufgabe 5: Python: einfacher Agent-Loop

Umfang: 9 Punkte

Abgabe: lauffähiger Python-Code plus kurze Erklärung

Implementieren Sie einen kleinen Agent-Loop in Python. Der Agent erhält das Ziel, einen Umsatzrückgang in der Schweiz zu erklären. Er soll das Ziel in Teilaufgaben zerlegen, passende simulierte Tools aufrufen, die Ergebnisse im Kontext speichern und daraus eine kurze Abschlussantwort erzeugen.

Erweitern Sie den Starter-Code so, dass mindestens zwei Tools genutzt werden. Der Agent soll zuerst Umsatzdaten abrufen, danach CRM-Notizen prüfen und anschliessend eine begründete Antwort formulieren. Der Code muss keine echte KI verwenden; die agentische Logik entsteht durch Planung, Tool-Auswahl, Kontextsammlung und Ergebnisbewertung.

Kommentieren Sie wichtige Codezeilen kurz. Nach dem Code erklären Sie in 5 bis 8 Sätzen, warum es sich um einen einfachen agentischen Loop handelt und welche Elemente für einen produktiven BI-Agenten noch fehlen würden.

Starter-Code

```
goal = "Erkläre Umsatzrückgang Schweiz"
context = []

def tool_sales_data(region):
    return {"region": region, "umsatz_alt": 120000, "umsatz_neu": 98000}

def tool_crm_notes(region):
    return ["Zwei Grosskunden haben Bestellungen verschoben",
            "Neü Konkurrenzaktion im Markt"]

def plan(goal, context):
    # TODO: Geben Sie eine Liste von Schritten zurück.
    return []

def run_step(step):
    # TODO: Wählen Sie passend zum Schritt ein Tool aus.
    return None

done = False
```

```

while not done:
    steps = plan(goal, context)
    for step in steps:
        result = run_step(step)
        context.append(result)
    done = Trü

print("Kontext:", context)
# TODO: Erzeugen Sie eine kurze, begründete Abschlussantwort.

```

Aufgabe 6: Tool-Design für BI-Agenten

Umfang: 7 Punkte

Abgabe: Tool-Spezifikation in Tabellenform plus kurzer Entscheidungslogik

Entwerfen Sie drei Tool-Schnittstellen für einen BI-Agenten: ein SQL-Tool, ein Dokumentensuche-Tool und ein Python-Analyse-Tool. Beschreiben Sie die Tools so konkret, dass ein Entwicklungsteam daraus eine erste Implementierung ableiten könnte.

Für jedes Tool sollen Sie Input, Output, erlaubte Nutzung, Grenzen, typische Fehler und Sicherheitsmassnahmen angeben. Beim SQL-Tool geht es zum Beispiel um erlaubte Tabellen, KPI-Definitionen, Lesezugriff und Qüry-Limits. Beim Dokumentensuche-Tool geht es um Qüllen, Aktualität, Ranking und Qüllenangaben. Beim Python-Tool geht es um Berechnungen, Visualisierungen, Laufzeitlimits und sichere Ausführung.

Ergänzen Sie abschliessend, wie der Agent entscheiden soll, welches Tool in welcher Situation verwendet wird.

Aufgabe 7: RAG versus Agentic RAG

Umfang: 7 Punkte

Abgabe: zwei Teile: Begriffsvergleich und Anwendung auf die BI-Frage

Erklären Sie den Unterschied zwischen traditionellem RAG und agentischem Retrieval. Traditionelles RAG ruft passende Dokumente ab und formuliert daraus eine Antwort. Agentic RAG geht weiter: Der Agent bewertet die ersten Treffer, erkennt veraltete oder unvollständige Qüllen, verfeinert die Suche und schliesst Informationslücken aktiv.

Wenden Sie diesen Unterschied auf die BI-Frage an: 'Warum ist die Marge in DACH gefallen?' Beschreiben Sie zürst, wie ein traditionelles RAG-System vorgehen würde. Entwickeln Sie danach einen agentischen Retrieval-Prozess mit mehreren Such- und Prüfschritten. Berücksichtigen Sie strukturierte Kennzahlen, CRM-Notizen, Preis- oder Rabattinformationen, Retouren, Lieferkosten und mögliche Marktinformationen.

Die Antwort soll zeigen, welche Qüllen der Agent zürst prüft, wann er seine Suchstrategie anpasst und wie er am Ende die Belastbarkeit der Antwort einschätzt.

Aufgabe 8: Python: KPI-Anomalie-Agent

Umfang: 10 Punkte

Abgabe: lauffähiger Python-Code plus Beispielausgabe

Programmieren Sie einen einfachen KPI-Anomalie-Agenten. Der Agent soll prüfen, ob die Conversion Rate von Woche 17 auf Woche 18 auffällig stark gefallen ist. Wenn eine Anomalie vorliegt, soll er eine wahrscheinliche Ursache aus einer kleinen Wissensbasis ableiten und eine konkrete Empfehlung für das Management formulieren.

Ergänzen Sie die Funktionen detect_anomaly, find_cause und recommend. Berechnen Sie den relativen Rückgang der Conversion Rate. Nutzen Sie eine einfache Regel: Wenn die mobile Fehlerquote deutlich höher ist als die Desktop-Fehlerquote, soll der Agent die Ursache 'mobile_error' priorisieren. Die Empfehlung soll nicht allgemein bleiben, sondern eine konkrete nächste Massnahme enthalten.

Geben Sie am Ende eine Beispielausgabe an. Erklären Sie kurz, warum der Agent nicht direkt eine operative Aktion ausführen sollte, sondern zunächst eine Empfehlung vorbereitet.

Starter-Code

```
kpis = {
    "conversion_week_17": 0.042,
    "conversion_week_18": 0.031,
    "mobile_error_rate_week_18": 0.18,
    "desktop_error_rate_week_18": 0.03,
}

knowledge_base = [
    {"signal": "mobile_error", "cause": "Checkout-Update verursacht Fehler auf mobilen Geräten"},
    {"signal": "campaign_end", "cause": "Marketingkampagne wurde beendet"},
]

def detect_anomaly(kpis):
    # TODO: Berechnen Sie den relativen Rückgang der Conversion.
    # Geben Sie Trü zurück, wenn der Rückgang grösser als 20% ist.
    return False

def find_cause(kpis, knowledge_base):
    # TODO: Nutzen Sie eine einfache Regel, um die wahrscheinlichste Ursache zu finden.
    return "unbekannt"

def recommend(cause):
    # TODO: Formulieren Sie eine konkrete Management-Empfehlung.
    return ""

if detect_anomaly(kpis):
    cause = find_cause(kpis, knowledge_base)
    print(recommend(cause))
else:
    print("Keine starke Anomalie erkannt.")
```

Aufgabe 9: Semantic Layer und KPI-Governance

Umfang: 7 Punkte	Abgabe: Tabelle mit drei KPIs plus kurze Reflexion
-------------------------	--

Entwickeln Sie für drei KPIs eine Mini-Semantik. Wählen Sie beispielsweise Umsatz, Marge, Churn, Customer Acquisition Cost, Lifetime Valü, Conversion Rate oder aktiver Kunde.

Geben Sie für jeden KPI Name, fachliche Definition, Formel, Granularität, erlaubte Filter, Datenquelle und typische Fehlinterpretation an. Die Definition soll so präzise sein, dass zwei Teams denselben KPI gleich berechnen würden. Beschreiben Sie auch, welche Folgen entstehen können, wenn ein Agent mit uneinheitlichen KPI-Definitionen arbeitet.

Schliessen Sie mit einer kurzen Reflexion ab: Warum ist der Semantic Layer für agentische BI wichtiger als für ein rein manuelles Dashboard?

Aufgabe 10: Human-in-the-Loop und Verantwortung

Umfang: 7 Punkte

Abgabe: Policy-Matrix mit Begründung

Ein BI-Agent darf Forecasts kommentieren, Risiken zusammenfassen und Aufgaben im Ticketing-System vorbereiten. Gleichzeitig können seine Ergebnisse Einfluss auf Budget, Preise, Kundenkommunikation oder interne Verantwortlichkeiten haben.

Erstellen Sie eine Policy-Matrix mit drei Kategorien: automatisch erlaubt, nur mit menschlicher Freigabe erlaubt und verboten. Ordnen Sie mindestens zehn Aktionen ein. Beispiele sind: interne Zusammenfassung erstellen, SQL-Abfrage starten, Ticket vorschlagen, Ticket automatisch erstellen, Forecast anpassen, externe E-Mail senden, Preisreduktion empfehlen, Preisreduktion ausführen, personenbezogene Daten auswerten oder Budget umschichten.

Begründen Sie Ihre Einordnung. Zeigen Sie dabei, wo Effizienzgewinne möglich sind und wo Verantwortung, Compliance oder Reputationsrisiken eine menschliche Freigabe erforderlich machen.

Aufgabe 11: Single-Agent oder Multi-Agent?

Umfang: 7 Punkte

Abgabe: maximal 1 Seite mit Architekturentscheidung und Begründung

Ein CFO möchte jeden Montagmorgen eine begründete Management-Zusammenfassung. Sie soll Kennzahlen, wesentliche Veränderungen, Risiken, mögliche Ursachen und konkrete To-dos enthalten. Entscheiden Sie, ob dafür ein Single-Agent oder eine Multi-Agent-Architektur geeigneter ist.

Bewerten Sie die Entscheidung anhand von Komplexität, Latenz, Spezialisierung, Debugging, Kosten, Nachvollziehbarkeit und Risiko. Wenn Sie eine Multi-Agent-Architektur wählen, skizzieren Sie die Rollen und Übergaben, zum Beispiel Research Agent, SQL Agent, Analytics Agent, Reviewer Agent und Manager Agent. Wenn Sie einen Single-Agent wählen, erklären Sie, wie dieser trotzdem kontrolliert und nachvollziehbar arbeitet.

Schliessen Sie mit einer Empfehlung ab, wie man klein starten und später skalieren könnte.

Aufgabe 12: Python: Mini-Retrieval-Agent

Umfang: 10 Punkte

Abgabe: lauffähiger Python-Code plus kurze Erklärung

Bauen Sie ein sehr einfaches Retrieval-System in Python und erweitern Sie es um ein agentisches Verhalten. Das System soll Dokumente nach Relevanz durchsuchen. Danach soll der Agent prüfen, ob die gefundenen Dokumente aktuell genug sind. Wenn nur veraltete Dokumente gefunden werden, soll er die Suche verfeinern und aktuellere Dokumente bevorzugen.

Ergänzen Sie `retrieve`, `is_outdated` und `agentic_retrieve`. Die Suche darf sehr einfach sein: Ein Dokument gilt als Treffer, wenn ein Wort aus der Suchanfrage im Titel oder Text vorkommt. Die agentische Erweiterung besteht darin, dass der Agent nicht blind den ersten Treffer akzeptiert, sondern die Aktualität bewertet und bei Bedarf eine zweite Suche simuliert.

Erklären Sie nach dem Code kurz, worin der Unterschied zu normalem RAG besteht und warum Aktualität im BI-Kontext besonders wichtig ist.

Starter-Code

```
documents = [
    {"title": "Marge DACH 2023", "year": 2023, "text": "Marge sinkt wegen Lieferkosten."},
    {"title": "Marge DACH 2026", "year": 2026, "text": "Marge sinkt wegen Rabattaktionen und Retouren."},
    {"title": "CRM Notizen DACH 2026", "year": 2026, "text": "Grosskunde fordert höhere Rabatte."},
]

def retrieve(query, docs):
    # TODO: Sehr einfache Suche: Treffer, wenn ein Query-Wort im Text oder Titel vorkommt.
    return []

def is_outdated(doc, min_year=2025):
    # TODO: True, wenn das Dokument zu alt ist.
    return False

def agentic_retrieve(query):
    results = retrieve(query, documents)
    # TODO: Wenn nur veraltete Dokumente gefunden werden, Suche verfeinern.
    return results

for doc in agentic_retrieve("Marge DACH"):
    print(doc["title"], "-", doc["text"])
```

Aufgabe 13: Evaluation, Monitoring und stille Fehler

Umfang: 7 Punkte

Abgabe: strukturierte Checkliste plus zwei Fehlerbeispiele

Entwickeln Sie ein Evaluations- und Monitoringkonzept für einen produktiven BI-Agenten. Bewertet werden soll nicht nur die finale Antwort, sondern der gesamte Workflow: Planung, Tool-Aufrufe, Quellen, Kosten, Latenz, Berechtigungen, fachliche Korrektheit und menschliches Feedback.

Nennen Sie mindestens sechs Messgrößen und beschreiben Sie, wie diese gemessen werden können. Beispiele sind korrekte Tool-Auswahl, Anteil der Antworten mit Quellen, Fehlerquote bei KPI-Definitionen, durchschnittliche Kosten pro Anfrage, Latenz, abgelehnte Aktionen wegen fehlender Berechtigung, Anteil menschlicher Korrekturen oder Stabilität über Testfälle hinweg.

Beschreiben Sie zusätzlich zwei stille Fehler. Ein stiller Fehler ist eine formal überzeugende, aber fachlich falsche oder unvollständige Empfehlung. Erklären Sie für jedes Beispiel, warum der Fehler gefährlich ist und wie man ihn durch Tests, Reviews, Traces oder Datenqualitätschecks erkennen könnte.

Aufgabe 14: Python: einfacher Task-Agent für Terminplanung

Umfang: 12 Punkte

Abgabe: lauffähiger Python-Code plus kurze Reflexion

Programmieren Sie einen einfachen Task-Agenten für Terminplanung. Der Agent soll Terminanfragen priorisieren, freie Slots prüfen und Buchungsvorschläge erzeugen. Er soll keine echte Buchung in einem externen System auslösen, sondern nur einen Vorschlag vorbereiten.

Ergänzen Sie `prioritize`, `find_slot` und `propose_booking`. Dringende Anfragen sollen zuerst bearbeitet werden. Wenn der bevorzugte Slot frei ist, soll dieser vorgeschlagen werden; falls nicht, soll der erste freie Slot gesucht werden. Sobald ein Vorschlag erstellt wurde, markieren Sie den Slot im lokalen Kalender als reserviert, damit er nicht doppelt vorgeschlagen wird.

Erklären Sie nach dem Code in fünf Sätzen, wo Human-in-the-Loop eingebaut ist. Beschreiben Sie auch, welche Risiken entstehen würden, wenn der Agent Termine automatisch bestätigt, absagt oder externe Nachrichten verschickt.

Starter-Code

```
reqüsts = [  
    {"name": "Anna", "urgent": Trü, "preferred": "Tü 10:00"},  
    {"name": "Ben", "urgent": False, "preferred": "Mon 09:00"},  
]  
  
calendar = {  
    "Mon 09:00": "free",  
    "Tü 10:00": "free",  
    "Wed 14:00": "busy",  
}  
  
def prioritize(reqüsts):  
    # TODO: Dringende Anfragen zürst sortieren.  
    return reqests  
  
def find_slot(reqüst, calendar):  
    # TODO: Preferred Slot nutzen, falls frei; sonst ersten freien Slot suchen.  
    return None  
  
def propose_booking(reqüst, slot):  
    # TODO: Geben Sie einen Buchungsvorschlag zurück, keine echte Buchung.  
    return ""  
  
for req in prioritize(reqüsts):  
    slot = find_slot(req, calendar)  
    print(propose_booking(req, slot))  
    # TODO: Markieren Sie den Slot als reserviert, falls ein Vorschlag erstellt wurde.
```